

LAB 4

Task # 01 (Creating a Parent Class (Super Class))

- Create a class named Animal with the following instance variables: name ,age.
- Create a method named makeSound() that prints "Generic animal sound".

CODE:

```
class Animal {  
  
    String name;  
  
    int age;  
  
    public void makeSound() {  
  
        System.out.println("Generic Animal Sound");  
  
    }  
}
```

```
public static void main(String[] args) {  
  
    }  
}
```

OUTPUT:

No output from this block of code.

Task # 02 (Creating a Child Class (Sub Class))

- Create a class named Dog that extends the Animal class.
- Override the makeSound() method in the Dog class to print "Woof!..".

CODE:

```
class Animal {  
  
    String name;  
  
    int age;  
  
    public void makeSound() {  
  
        System.out.println("Generic Animal Sound");  
  
    }  
  
    public static void main(String[] args) {  
  
        Dog dog = new Dog();  
  
        dog.makeSound();  
  
    }  
}
```

```
class Dog extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Woof!");  
  
    }  
}
```

OUTPUT:

Woof!

LAB 4

Task # 03 (Creating Another Child Class)

- Create a class named Cat that extends the Animal class.
- Override the makeSound() method in the Cat class to print "Meow!"

CODE:

```
class Animal {  
  
    String name;  
  
    int age;  
  
    public void makeSound() {  
  
        System.out.println("Generic Animal Sound");  
    }  
  
    public static void main(String[] args) {  
  
        Cat cat1 = new Cat();  
  
        cat1.makeSound();  
  
    }  
}  
  
class Cat extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Meow!");  
  
    }  
}
```

OUTPUT:



Meow!

LAB 4

Task # 04 (Creating Objects and Calling Methods)

- Create objects of the Dog and Cat classes.
- Call the makeSound() method on each object to demonstrate polymorphism.

CODE:

```
class Animal {  
  
    String name;  
  
    int age;  
  
    public void makeSound() {  
  
        System.out.println("Generic Animal Sound");  
    }  
  
    public static void main(String[] args) {  
  
        Dog dog1 = new Dog();  
  
        dog1.makeSound();  
  
        Cat cat1 = new Cat();  
  
        cat1.makeSound();  
    }  
}  
  
class Dog extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Woof!");  
    }  
}
```

```
class Cat extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Meow!");  
    }  
}
```

OUTPUT:



```
Woof!  
Meow!
```

LAB 4

Task # 05 (Understanding Inheritance Hierarchy)

- Create a class named GoldenRetriever that extends the Dog class.
- Explain how inheritance creates a hierarchical relationship between classes.
- Discuss the concept of single inheritance and multiple inheritance (which is not directly supported in Java).

CODE:

```
class Animal {  
  
    String name;  
  
    int age;  
  
    public void makeSound() {  
  
        System.out.println("Generic Animal Sound");  
    }  
  
    public static void main(String[] args) {  
  
        Dog dog1 = new Dog();  
  
        dog1.makeSound();  
  
        Cat cat1 = new Cat();  
  
        cat1.makeSound();  
    }  
}
```

```
class Dog extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Woof!");  
    }  
}
```

```
}  
  
class GoldenRetriever extends Dog {  
  
    GoldenRetriever() {  
  
        this.name = "GoldenRetriever";  
  
        this.age = 6;  
    }  
}  
  
class Cat extends Animal {  
  
    @Override  
  
    public void makeSound() {  
  
        System.out.println("Meow!");  
    }  
}
```

OUTPUT:

NO Output for this block of code.

LAB 4

Task #05 (b). Explain how inheritance creates a hierarchical relationship between classes.

Hierarchy means that classes are arranged in a structure where a subclass inherits properties or attributes and behaviors or functions from a superclass.

In our Example:

Golden-Retriever is a Dog and Dog is an animal.

or

A Golden-Retriever can be treated as a dog or as an Animal due to inheritance between them.

Task #05 (b). Discuss the concept of single inheritance and multiple inheritance (which is not directly supported in Java).

Single Inheritance refers to the concept where a class can inherit from only one direct superclass .Like in our example Golden Retriever inherits from dog and Dog inherits from Animal. This is an example of **Single Inheritance**.

Multiple Inheritance refers to the concept where a class can inherit from more than one class. This means a class could inherit properties and methods from multiple classes at the same time.

Java Programming language doesn't support **Multiple Inheritance** due to **Diamond Problem**. Basically a compiler issue, when the two super classes of inherited object have same name of methods leading to confusion or ambiguity about which method to call. But we can achieve similar functionality using the implementation of interfaces.